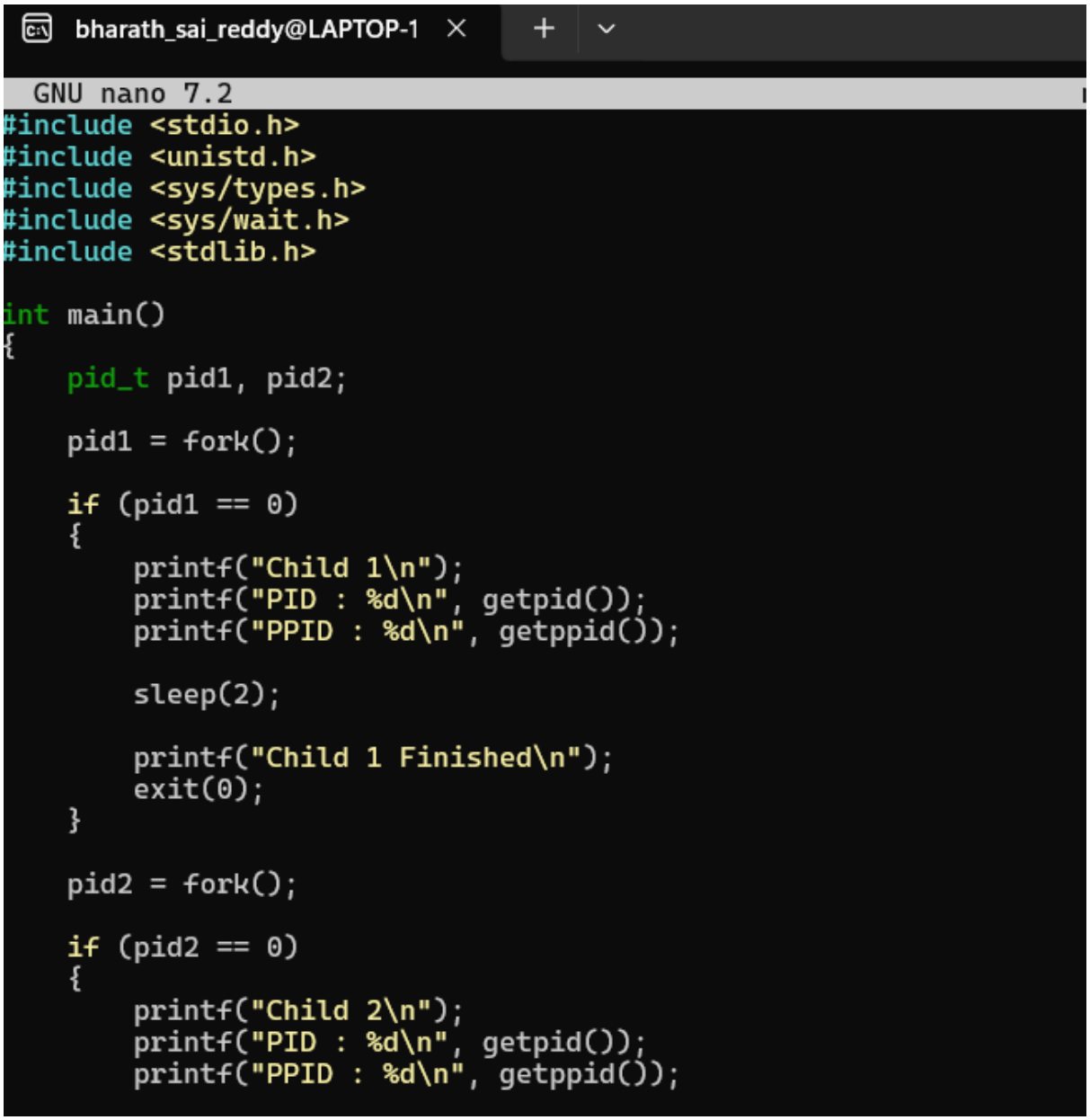
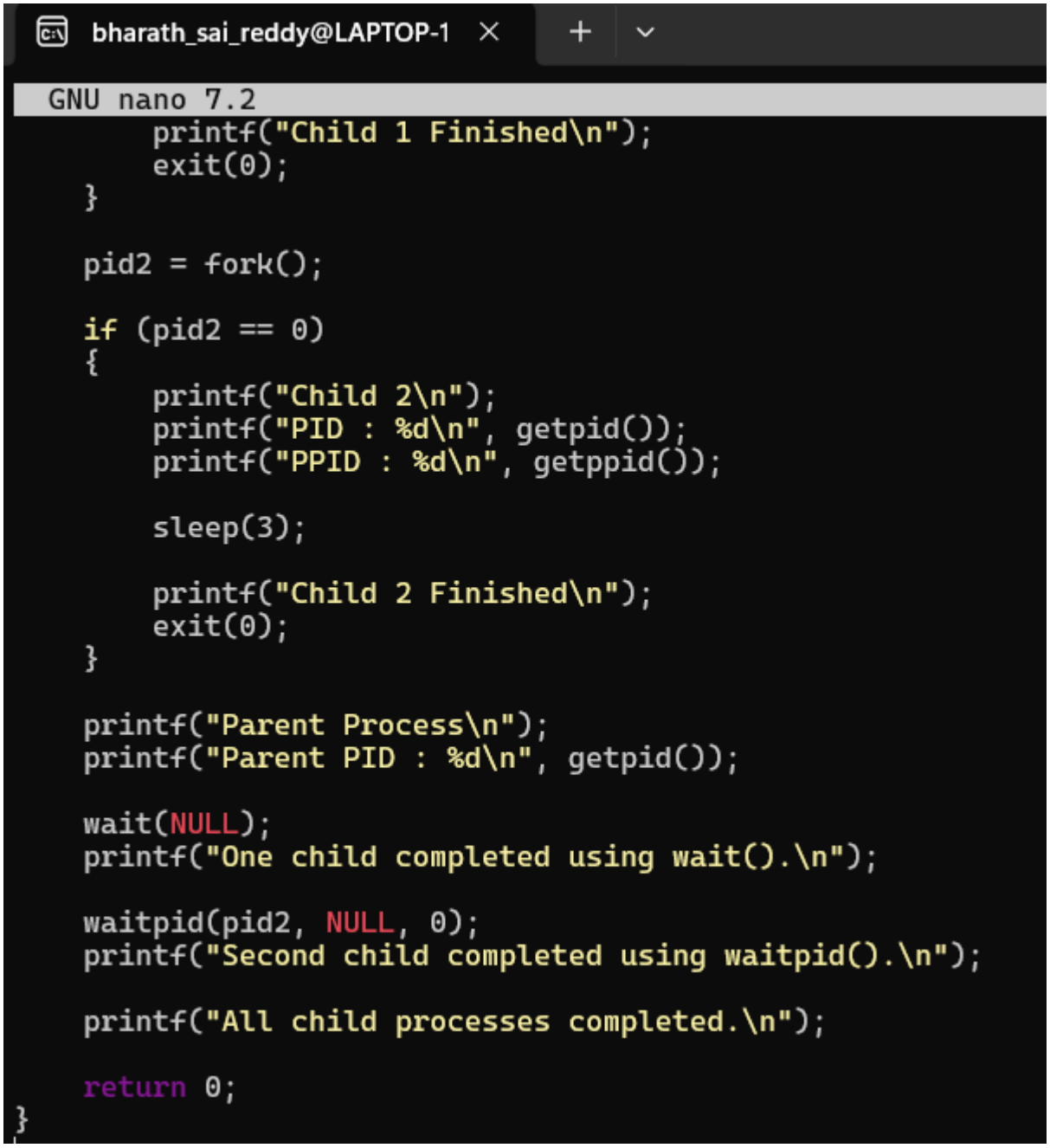
Practical -04

Code:




bharath_sai_reddy@LAPTOP-1
GNU nano 7.2
printf("Child 1 Finished\n");
exit(0);
}
pid2 = fork();
if (pid2 == 0)
{
printf("Child 2\n");
printf("PID : %d\n", getpid());
printf("PPID : %d\n", getppid());
sleep(3);
printf("Child 2 Finished\n");
exit(0);
}
printf("Parent Process\n");
printf("Parent PID : %d\n", getpid());
wait(NULL);
printf("One child completed using wait().\n");
waitpid(pid2, NULL, 0);
printf("Second child completed using waitpid().\n");
printf("All child processes completed.\n");
return 0;
}

bharath_sai_reddy@LAPTOP-1

GNU nano 7.2

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Sleeping for 20 seconds...\n");

        sleep(20);

        printf("Parent exiting.\n");
    }

    return 0;
}
```

1)

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/Practical-04$ ./multiple_children
Child 2
Child 1
PID : 1345
PPID : 1343
Parent Process
PID : 1344
PPID : 1343
Parent PID : 1343
Child 1 Finished
One child completed using wait().
Child 2 Finished
Second child completed using waitpid().
All child processes completed.
```

2)

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/Practical-04$ ./zombie_process
Parent Process
PID : 1353
Sleeping for 20 seconds...
Child Process
PID : 1354
Child exiting...
Parent exiting.
```

3)

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/Practical-04$ ./zombie_fixed
Parent Process
PID : 1385
Child Process
PID : 1386
Child exiting...
Child completed successfully.
No zombie process remains.
Parent exiting.
```